UNIVERSIDAD DON BOSCO

DIRECCIÓN DE EDUCACIÓN A DISTANCIA

Diseño y Programación de Software Multiplataforma DPS941 G01T (Virtual)

DOCENTE:

Alexander Alberto Siguenza Campos

EXPERIENCIA DE APRENDIZAJE 3

Foro 2: Aplicación de Control de Gastos Personales en React Native

GRUPO #6

Integrantes:

Victor Emmanuel Velasco Martinez	VM251307
Fernando José Gómez Hernández	GH251230
José Eduardo Aquino Medrano	AM252078
Milton Adalberto Ayala Ventura	AV251709
William Eduardo Montano Aguilar	MA251192

ENLACE DE REPOSITORIO:

https://github.com/VMz69/cgp-app.git

VIDEO DEMOSTRACION:

https://www.youtube.com/watch?v=6irFKT40LMg

FECHA DE ENTREGA:

10 de mayo del 2026

Indice

Contenido

Indice 1
2. Introducción 2
2.1 Descripción de la app 2
2.2 objetivo del proyecto 2
3. Investigación Firebase Auth 2
3.1 Firebase Authentication: 2
3.2 Tipos: email/password, Google 3
3.3 Ventajas 3
3.4 Desventajas 3
3.5 Casos de uso 3
4. Implementación técnica 4
4.1 Creación del proyecto en Firebase 4
4.2 Habilitar Authentication 4
4.3 Configuración de Google provider 4
4.4 Configuración de Firestore 4
4.5 Conexión con React Native 5
5. Desarrollo de la app 5
5.1 Login email 5
5.2 Login Google 5
5.3 Creación de gastos 6
5.4 Estructura en Firestore 6
5.5 Cálculo mensual 7
6. Capturas de la app móvil 7
6.1 Login 8
6.2 Home 8
6.3 Agregar gasto 9
6.4 Historial 9
6.5 Firebase console 10
7. Conclusiones 11
7.1 Aprendizajes 11
7.2 Dificultades del proyecto 12
8. Referencias 13

2. Introducción

2.1 Descripción de la app

CGP App (Control de Gastos Personales) es una solución móvil multiplataforma desarrollada bajo el ecosistema de React Native y Expo. La aplicación permite a los usuarios gestionar de manera integral su economía personal a través de un sistema de registro de ingresos y egresos en tiempo real.
La herramienta destaca por su interfaz intuitiva que facilita operaciones de creación, edición y eliminación de registros financieros (CRUD). Gracias a su integración con Firebase, la aplicación ofrece persistencia de datos en la nube y un sistema de autenticación seguro que permite el acceso mediante correo electrónico o cuentas de Google. El diseño está optimizado para ofrecer un resumen financiero inmediato, mostrando métricas clave como el gasto acumulado mensual y el historial general de transacciones.

2.2 objetivo del proyecto

El propósito fundamental de este proyecto es proporcionar al usuario una herramienta tecnológica eficiente que fomente la disciplina financiera y el control de gastos cotidianos. De manera específica, el proyecto persigue:

- Centralización de Datos al implementar una base de datos NoSQL con Cloud Firestore para almacenar de forma jerárquica y segura cada transacción realizada.
- Accesibilidad y Movilidad al garantizar que el usuario pueda registrar sus gastos desde cualquier lugar mediante una aplicación móvil ligera ejecutada a través de Expo Go.
- Optimización de Recursos al desarrollar el sistema en un entorno de ingeniería eficiente, utilizando una Lenovo Flex con procesador Ryzen 5 debidamente optimizada para priorizar el rendimiento del empaquetador Metro Bundler y el flujo de desarrollo.
- Análisis Financiero al automatizar el cálculo de sumatorias mensuales para que el usuario posea una visión clara de su capacidad de gasto y ahorro en tiempo real.

3. Investigación Firebase Auth

3.1 Firebase Authentication:

Es un servicio de autenticación en la nube que permite a los desarrolladores implementar sistemas de inicio de sesión de forma segura y sencilla. Proporciona SDKs fáciles de usar y bibliotecas de interfaz de usuario listas para usar, encargándose de la gestión de identidades, el almacenamiento de usuarios y la seguridad de las credenciales.

3.2 Tipos: email/password, Google

Firebase ofrece una solución omnicanal que incluye:

- Correo y contraseña: Registro tradicional con validación de email.
- Proveedores Federados: Inicio de sesión con Google, Facebook, GitHub, Apple, Twitter, entre otros.
- Autenticación por teléfono: Uso de SMS para validación de identidad.
- Anónimo: Permite usar la app sin registrarse inicialmente, pudiendo vincular los datos a una cuenta real después.

3.3 Ventajas

Entre las principales ventajas de su uso se encuentran los siguientes:

- Facilidad de implementación: Reduce drásticamente el tiempo de desarrollo del backend de seguridad.
- Escalabilidad: Al ser parte de la infraestructura de Google, maneja millones de usuarios sin configuración adicional.
- Seguridad: Cumple con estándares internacionales de protección de datos y manejo de sesiones.

3.4 Desventajas

Principales desventajas de su uso, son los siguientes:

- Dependencia del proveedor (Vendor Lock-in): Migrar a otra plataforma de autenticación puede ser complejo una vez que la base de usuarios es grande.
- Personalización limitada: Aunque es flexible, ciertos flujos de seguridad muy específicos pueden ser difíciles de modificar.
- Costo: Aunque tiene una capa gratuita generosa, los servicios como SMS tienen costos asociados tras superar ciertos límites.

3.5 Casos de uso

Algunas aplicaciones para esta tecnología son los siguientes:

- Aplicaciones Móviles y Web: Para control de acceso y personalización de perfiles.
- Plataformas de E-commerce: Para gestionar carritos de compra vinculados a un usuario específico.

- Sistemas de Gestión Interna: Para autenticar empleados mediante cuentas corporativas de Google.

4. Implementación técnica

4.1 Creación del proyecto en Firebase

Se configuró un nuevo proyecto en la consola de Firebase específicamente para el entorno Android de la aplicación. En este paso, se registró el paquete del proyecto y se descargó el archivo de configuración google-services.json. Este archivo es fundamental para que el SDK de Firebase reconozca la aplicación y permita la comunicación con los servicios de nube.

4.2 Habilitar Authentication

Se activó el módulo de Authentication para gestionar la identidad de los usuarios. En la consola, se habilitaron los métodos de acceso "Correo electrónico/contraseña" y "Google". Esta configuración permite que la aplicación gestione sesiones persistentes, permitiendo que el usuario mantenga su sesión activa entre ingresos y egresos de la app.

4.3 Configuración de Google provider

Debido a que el proyecto utiliza Expo Go para el desarrollo, la configuración de Google se realizó mediante OAuth 2.0 con Expo Auth Session. Esto implica configurar una redirección web (exp://) para que, al autenticarse, Google devuelva el control a la aplicación móvil de forma segura. Se vincularon los IDs de cliente de Google Cloud para autorizar las peticiones desde el emulador y el dispositivo físico android.

4.4 Configuración de Firestore

Para la gestión de los datos financieros, se habilitó Cloud Firestore en modo de producción, estableciendo una base de datos NoSQL orientada a documentos. El proceso de configuración técnica consistió en:

- Definición de Reglas de Seguridad: Se implementaron reglas de acceso basadas en la identidad del usuario (request.auth != null). Esto garantiza que los datos de ingresos y egresos sean privados, permitiendo operaciones de lectura, escritura y eliminación únicamente si el UID del documento coincide con el UID del usuario autenticado.

- Creación de la Colección: Se inicializó la colección raíz denominada gastos. A diferencia de las bases de datos relacionales, Firestore permite una estructura flexible donde cada documento se crea dinámicamente al realizar el primer registro desde la app.

- Configuración del SDK en el Proyecto: Se integró el servicio dentro del archivo de configuración de Firebase en el código fuente, exportando la instancia db (Firestore) para ser utilizada por los componentes de React Native.
- Índices y Consultas: Se verificó la propagación de datos para permitir consultas filtradas por fecha y usuario, lo cual es vital para el rendimiento del "Cálculo Mensual" descrito posteriormente en el desarrollo.

4.5 Conexión con React Native

La integración técnica se basó en el ecosistema Expo Router, utilizando la estructura de directorios app/.

- Rutas de Autenticación: Ubicadas en (auth)/, gestionan el login y registro.
- Rutas Principales: Ubicadas en (tabs)/, contienen la lógica de negocio para visualizar el balance y agregar nuevos movimientos.
- Comunicación: Se instalaron las dependencias necesarias mediante (npm instal)l y se configuró el archivo (_layout.tsx) como el Router principal para manejar el flujo de navegación entre el login y la pantalla principal de la aplicación.

5. Desarrollo de la app

5.1 Login email

La autenticación base permite a los usuarios acceder a su información financiera de forma privada mediante el módulo Firebase Auth.

- Interfaz: Se diseñó un formulario con campos para correo electrónico y contraseña, incluyendo validación de credenciales en tiempo real.
- Flujo: Al presionar "Entrar", el SDK de Firebase valida los tokens de acceso y redirige al usuario a la pantalla de "Resumen".
- Seguridad: Las contraseñas son gestionadas y cifradas por Google, garantizando que los datos sensibles no se almacenen de forma insegura en el dispositivo.

5.2 Login Google

Se implementó un método de acceso rápido para mejorar la experiencia de usuario (UX).

- Integración: Utiliza el proveedor de identidad de Google, permitiendo que el usuario se autentique sin crear una contraseña nueva.
- Sincronización: Al detectar una cuenta de Google, la app asocia automáticamente el perfil con el ID de usuario en la Firebase Console.

5.3 Creación de gastos

Este módulo es el núcleo de la aplicación y permite la persistencia de datos financieros.

- Formulario de Registro: Incluye campos obligatorios para el Nombre del gasto, Categoría (Comida, Otros, etc.), Fecha y Monto.
- Experiencia de Usuario: Se utilizan selectores desplegables y campos con valores por defecto (como la fecha actual) para agilizar la entrada de datos.
- Feedback: Al guardar, el sistema envía los datos a la nube y actualiza instantáneamente el historial del usuario.

5.4 Estructura en Firestore

Para la base de datos, se eligió Cloud Firestore debido a su capacidad de respuesta en tiempo real y estructura NoSQL.

- Colección principal: gastos.
- Documentos: Cada gasto es un objeto que contiene los siguientes atributos:
    - nombre: String (ej. "pollito2").
    - categoría: String (ej. "Comida").
    - monto: Number (ej. 25.00).
    - fecha: String (formato AAAA-MM-DD).
    - userId: String, UUID del usuario para asegurar que solo él vea sus propios datos.
    - createdAt: any, fecha de creación

5.4.1 Operaciones CRUD

- **Create**: Para crear un nuevo registro, se utiliza la función addDoc. Esta función recibe la referencia de la colección y un objeto con los datos. Firestore genera automáticamente un ID alfanumérico único para cada documento.

```
await addDoc(collection(db, "expenses"), {
name,
amount,
category,
date,
userId: user.uid, // Por la relación con el usuario autenticado
createdAt: new Date(),
});
```

- **Read:** La lectura se realiza mediante una consulta (query) filtrada por el userId para traer solo los datos del usuario. Se utiliza getDocs para obtener una instantánea de la colección.

```
const q = query(
collection(db, "expenses"),
where("userId", "==", user.uid)
);
const querySnapshot = await getDocs(q);
```

- **Update:** Para modificar un gasto existente, se obtiene una referencia al documento específico mediante su ID y se utiliza updateDoc enviando únicamente los campos que han cambiado (actualización parcial).

```
const docRef = doc(db, "expenses", id);
await updateDoc(docRef, { ...data });
```

- **Delete:** La eliminación se ejecuta apuntando al ID único del documento visualizado en la consola de Firebase. Con el borrado, no estamos haciendo un “softDelete”. Cada documento dentro de la colección es realmente **eliminado** de la base de datos. Mediante el método doc(db, "colección", "id"), apuntamos al nodo de datos en el servidor. La función deleteDoc envía una instrucción de borrado a Firestore. Esta operación es permanente y una vez confirmada, los datos no pueden recuperarse.

```
const docRef = doc(db, "expenses", id);
await deleteDoc(docRef);
```

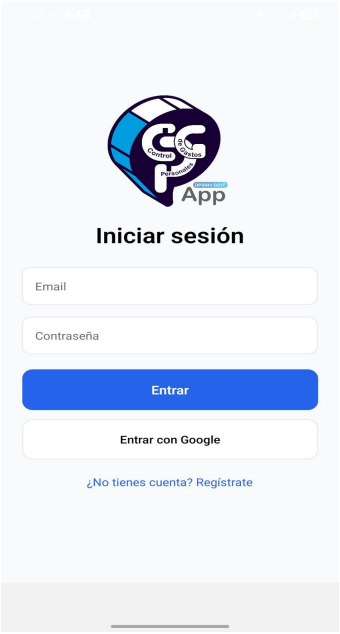
5.5 Cálculo mensual

La aplicación procesa los datos almacenados para ofrecer una visión financiera clara al usuario en la pantalla de Inicio.

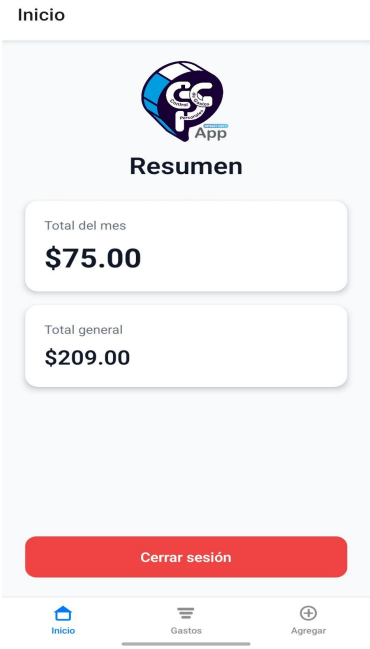
- Total, del Mes: Se aplica un filtro de servidor en Firestore para sumar únicamente los montos cuya fecha coincida con el mes en curso (ej. $75.00).
- Total, General: Sumatoria histórica de todos los registros asociados a la cuenta (ej. $209.00).
- Lógica de Negocio: Estos cálculos se realizan del lado del cliente (React Native) tras recibir la respuesta de la consulta a Firebase, asegurando que el resumen esté siempre actualizado.

6. Capturas de la app móvil

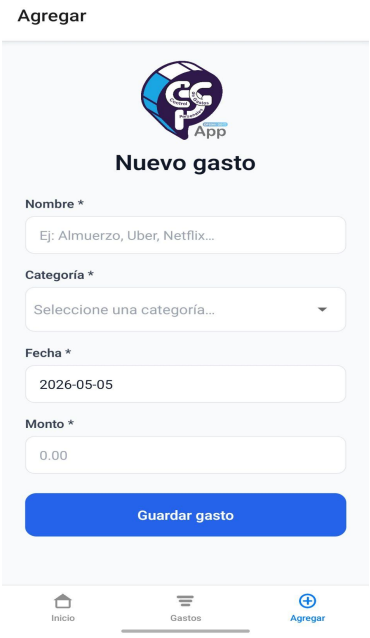
6.1 Login



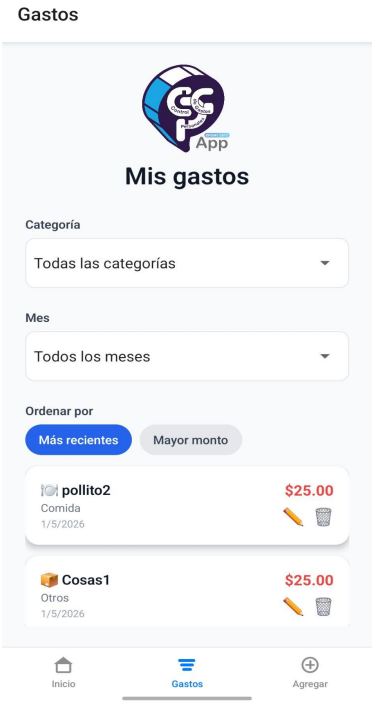
6.2 Home



6.3 Agregar gasto



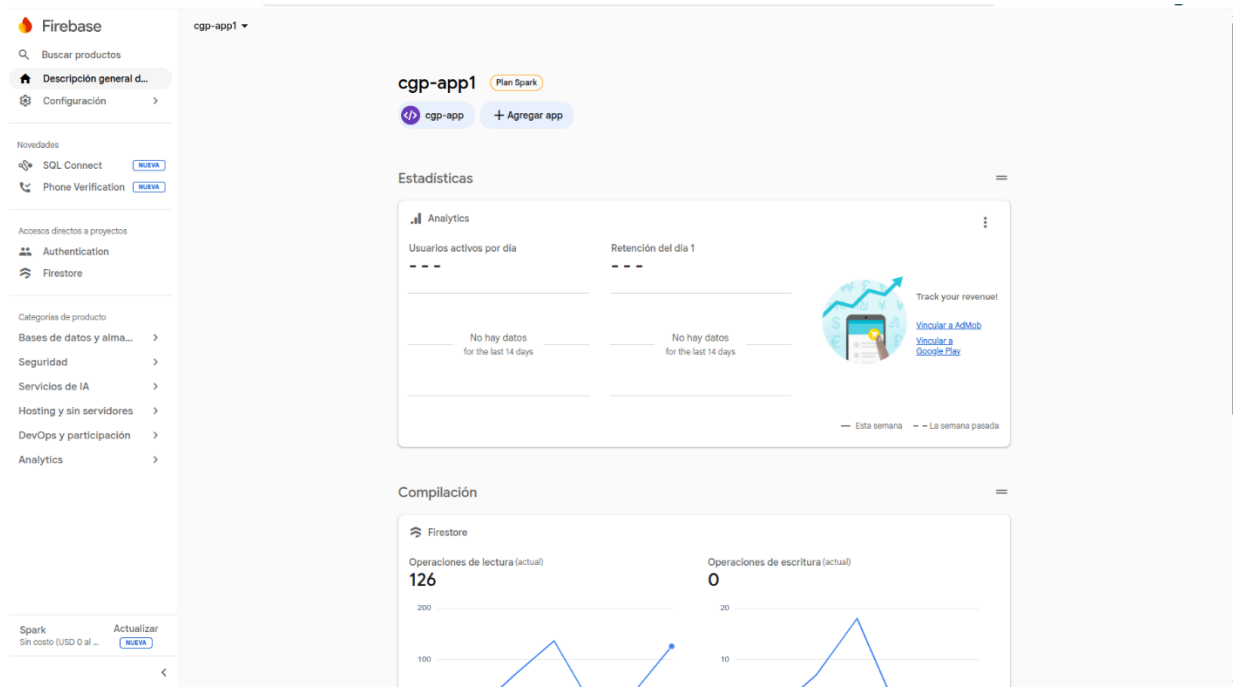
6.4 Historial



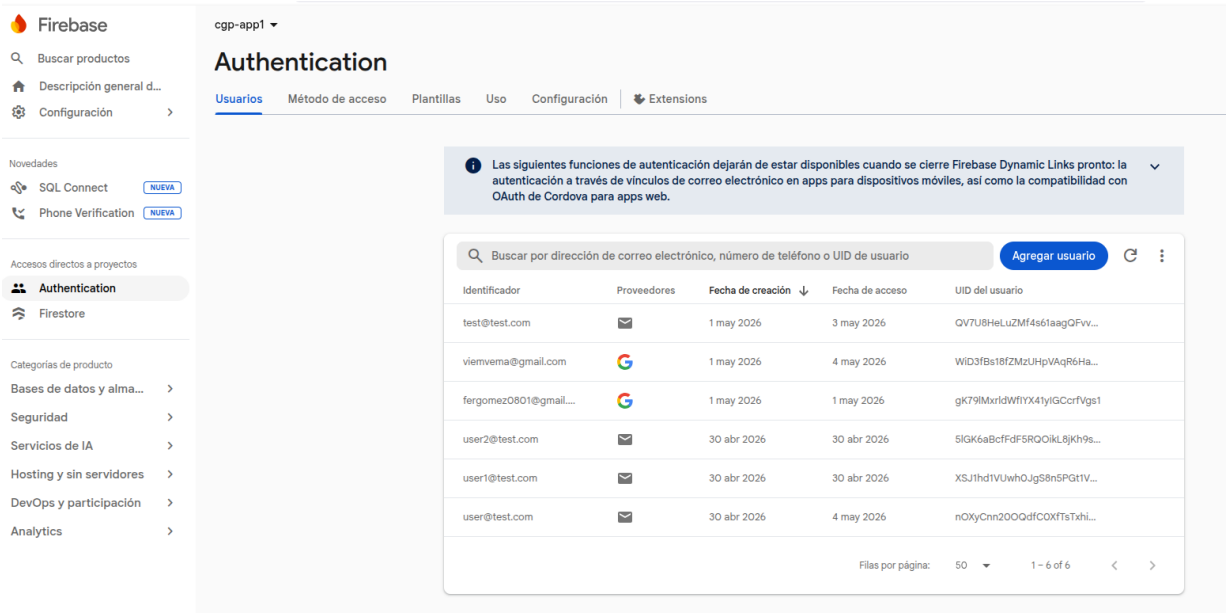
6.5 Firebase console

Para garantizar la persistencia de datos y la gestión de usuarios en la aplicación cgp-app, se ha integrado Google Firebase como plataforma de Backend-as-a-Service (BaaS).

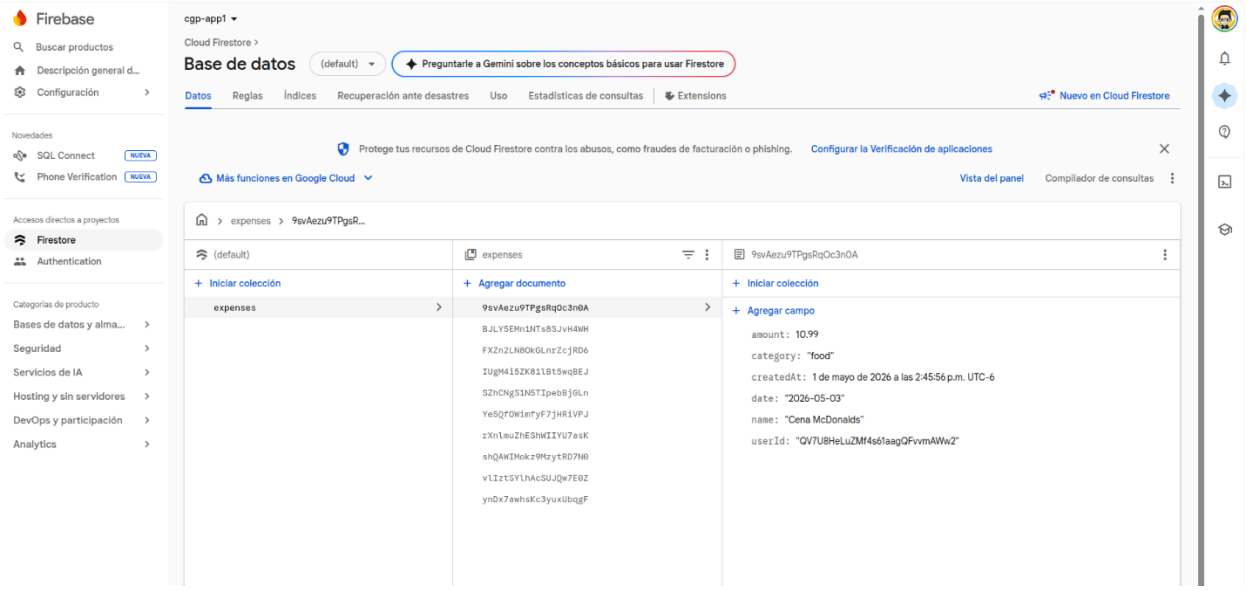
Ventana principal de proyecto Firebase:



FireBase Authentication:



Firestore:



7. Conclusiones

1. Se logró consolidar un ecosistema de desarrollo moderno utilizando React Native y Expo para la interfaz móvil, junto con Firebase para la gestión de datos en la nube.
2. La aplicación cumple con su objetivo primordial de centralizar los gastos, permitiendo visualizar de forma clara el total del mes y el total general, lo que facilita la toma de decisiones financieras.
3. El sistema permite realizar el ciclo completo de operaciones (CRUD): creación, lectura, actualización y eliminación de registros de gastos, garantizando la integridad de la información mediante confirmaciones de seguridad.
4. Mediante la implementación de Firebase Authentication, se garantiza que solo usuarios autorizados (vía correo o Google) puedan acceder y gestionar su información persona

7.1 Aprendizajes

1. Se comprendió la importancia de configurar correctamente el SDK de Firebase (API Keys, Project IDs) para establecer un puente de comunicación seguro entre el entorno local y los servicios en la nube.
2. El uso de Expo Go permitió entender el flujo de despliegue en dispositivos físicos, facilitando el debugging en tiempo real sin depender exclusivamente de emuladores pesados, así se maximiza los recursos de nuestras estaciones de trabajo.

7.2 Dificultades del proyecto

a) Limitación Crítica utilizando Expo Go cuando se requiere autenticación nativa dentro de la app. Ya que Expo Go es una aplicación contenedora genérica. Esto significa que viene con un conjunto fijo de bibliotecas nativas pre-instaladas, por lo tanto, no puedes instalar bibliotecas nativas personalizadas (como @react-native-google-signin/google-signin) dentro de Expo Go. Si tu código intenta llamar a una función de Google que requiere código nativo no incluido en Expo Go, la aplicación fallará.

b) Inestabilidad del OAuth vía Navegador debido a que Firebase permite la autenticación web, al usar expo-auth-session (el método estándar en Expo), pero nos presentó problemas de redirección, por lo que resulto que en varios intentos de inicio de sesión en Android nos resultó en un error DISMISS debido a fallos en el sistema de deep links de Android.

8. Referencias

Google. (s.f.). *Firebase Authentication*. Firebase. https://firebase.google.com/docs/auth

Gupta, A., & Goyal, A. (2020). *Practical Guide to Firebase: Building Real-time Applications*. BPB Publications. Google Books - Practical Guide to Firebase

Moroney, L. (2017). *The Definitive Guide to Firebase: Build Real-Time Applications with Firebase*. Apress. SpringerLink (Editorial Apress)

Wadhwa, G. (2023). *Full Stack Development with Google Firebase*. O'Reilly Media. O'Reilly Media - Full Stack Development with Google Firebase

Expo. (s.f.). *Expo Router: File-based routing*. Expo Documentation. https://docs.expo.dev/router/introduction/

Google. (s.f.). *Firebase Authentication with Google*. Firebase Docs. https://firebase.google.com/docs/auth/web/google-signin

Invertase. (s.f.). *React Native Firebase - Getting Started*. RNFirebase. https://rnfirebase.io/

Google. (s.f.). Material Design 3: The adaptable design system. Material Design. https://m3.material.io/

Expo. (s.f.). *Expo Go: Fast, simple, and powerful development tools*. Expo Documentation. https://docs.expo.dev/get-started/expo-go/

Clerk. (2026). *Native vs. Browser OAuth in Expo: A Decision Guide for Social Login*. Recuperado el 5 de mayo de 2026, de https://clerk.com/articles/native-vs-browser-oauth-in-expo

Expo Documentation. (s.f.). *Introduction to development builds*. Recuperado el 5 de mayo de 2026, de https://docs.expo.dev/develop/development-builds/introduction/

Haru12255221. (2026, enero 22). *Expo SDK 54 + EAS. Recuperado el 5 de mayo de 2026, de https://qiita.com/haru12255221/items/cf113892eda163b33fd0